



Obligatorio Marzo 2026 Estructura de Datos y Algoritmos 2

Integrantes:

Clase: M4A



Sebastián Quiroz

323189



Juan Francisco Zangaro

326706

Índice

Índice.....	2
Tabla de ejercicios.....	3
Ejercicio 1.....	4
TADs usado.....	4
Flujo.....	4
Complejidad.....	4
Ejercicio 2.....	5
TADs usados.....	5
Flujo.....	5
Complejidad.....	5
Ejercicio 3.....	6
TADs usados.....	6
Flujo.....	6
Complejidad.....	7
Ejercicio 4.....	8
TADs usados.....	8
Cambios importantes para rendimiento.....	8
Flujo.....	9
Dijkstra.....	9
Complejidad.....	9
Ejercicio 5.....	10
TADs usados.....	10
Flujo.....	10
Declaración de autoría.....	12

Tabla de ejercicios

Ejercicio	Resultado
1	Completado
2	Completado
3	Completado
4	Completado
5	Completado

Ejercicio 1

El ejercicio procesa una secuencia de llaves y puertas. Cada par de caracteres representa:

- una llave encontrada
- una puerta que se intenta abrir

La solución mantiene una tabla hash con la cantidad disponible de llaves de cada letra. Cuando aparece una llave se incrementa su contador. Cuando aparece una puerta, se transforma a minúscula y se consulta si hay una llave disponible.

Si hay llave disponible, se consume una. Si no hay, se incrementa el contador de puertas que no se pudieron abrir.

TADs usado

hashC_table<char, int>: hash table cerrada donde la clave son los chars que representan a las posibles llaves (a, b, c, ..., z) y los valores son la cantidad de llaves que tiene cada char.

Como el conjunto de claves son letras, la tabla trabaja sobre un conjunto chico y fijo.

Flujo

1. Leer la cantidad de puertas.
2. Leer el string de entrada.
3. Recorrer el string de a dos caracteres.
4. Guardar la llave encontrada en la tabla.
5. Convertir la puerta a minúscula.
6. Ver si existe una llave disponible para esa puerta.
7. Si no existe, sumar una puerta no abierta.
8. Imprimir el total.

Complejidad

Si P es la cantidad de puertas, el string tiene $2P$ caracteres y el ciclo principal itera P veces.

Las operaciones de tabla son $O(1)$ promedio y, en este caso, prácticamente constantes porque hay pocas letras posibles.

Por ende, la complejidad es de $O(P)$

Ejercicio 2

El ejercicio calcula el ganador de un juego por rondas. Cada ronda tiene un jugador y una variación de puntaje.

La regla importante es que gana el jugador que:

1. termina con el puntaje final máximo
2. alcanza ese puntaje primero durante la reconstrucción ronda a ronda

Por eso, para obtener la solución, hacemos dos pasadas:

- primero calcula los puntajes finales
- después reconstruye los puntajes parciales para encontrar quien alcanza primero el máximo final

TADs usados

hashCdbIHash_table<string, Jugador>: hash table con doble hashing (requerido por letra), donde nuestro conjunto de claves son los nombres de los participantes (que entendemos que son únicos) y nuestros valores son el struct Jugador, los cuales tienen dos elementos ptsFinal y ptsParcial.

A su vez, se usan arrays auxiliares para poder reconstruir las rondas en el mismo orden que fueron leídas.

Flujo

1. Leer la cantidad de rondas.
2. Para cada ronda:
 - leer nombre y puntaje
 - guardar nombre y cambio en arrays
 - acumular el puntaje final del jugador en la tabla
 - guardar el nombre si es un jugador nuevo
3. Recorrer los distintos jugadores para obtener el puntaje final máximo.
4. Recorrer nuevamente las rondas en orden:
 - actualizar puntaje parcial del jugador
 - si su puntaje final es el máximo y su parcial ya alcanzó ese máximo, imprimirlo y terminar

Complejidad

Sea: R = cantidad de rondas, J = cantidad de jugadores distintos con $J \leq R$.

Las pasadas son: $O(R) + O(J) + O(R)$

Entonces, con hash promedio $O(1)$: $O(R)$

Ejercicio 3

El ejercicio ordena pacientes por prioridad de atención.

La prioridad se define con:

1. mayor urgencia primero
2. si empatan en urgencia, menor hora primero
3. si también empatan en hora, menor número de ingreso primero

Para eso se usa un max heap donde la prioridad está representada por el struct **PrioridadUrgencia**.

Lo más complicado de este ejercicio fue entender que teníamos que comparar. Luego de muchas ideas y de hablar con los profesores, llegamos a la conclusión de que era necesario redefinir los operadores mayor (>) e igualdad (==) dentro del struct **PrioridadUrgencia** para que el max heap pueda comparar los distintos nodos y acomodarlos correctamente. Para hacer overload de los operadores empezamos por un artículo que vimos en [GeeksforGeeks](https://www.geeksforgeeks.org/overloading-comparison-operators-cpp/) y luego nos ayudó Federico.

TADs usados

max_heap<Paciente, PrioridadUrgencia>: max heap donde la prioridad está definida por la **PrioridadUrgencia** y con objeto **Paciente**.

En base a **PrioridadUrgencia** se define cuál paciente tiene mayor prioridad.

El struct **Paciente** guarda:

- int nro
- int hora
- int urgencia
- int nroIngreso

Flujo

1. Leer la cantidad de pacientes.
2. Para cada paciente:
 - leer número, hora y urgencia
 - crear el paciente
 - crear su prioridad
 - insertarlo en el heap
3. Mientras el heap no este vacio:
 - sacar el maximo
 - imprimir el número del paciente

Complejidad

Sea: n = cantidad de pacientes

Insertar cada paciente cuesta: $O(\log n)$

Se hacen n inserciones: $O(n \log n)$

Luego se hacen n extracciones, también de costo $O(\log n)$: $O(N \log N)$

Tiempo total: **$O(n \log n)$**

Ejercicio 4

Para el ejercicio 4 modelamos planetas y portales como un grafo ponderado no dirigido.

Cada planeta llega como string, por lo que primero se le asigna un id numérico usando una tabla hash. Luego se agrega cada portal como una arista ponderada en una lista de adyacencia.

Para resolver el ejercicio se usa Dijkstra con cola de prioridad mínima, esto con el fin de encontrar el camino más corto entre el origen y el destino.

TADs usados

Se usan:

- **adj_list_graph**: Grafo representado por listas de adyacencia usado para resolver el problema.
- **hashCdbIHash_table<string, int>**: Hash table donde indexamos cada uno de los vértices. Cada vértice es un string con el formato *planetanúm*.
- **min_heap<int, int>** : Heap que utilizamos para hacer Dijkstra, asegurándonos de visitar el vértice adyacente con menor peso en la arista.

Flujo

1. Leer cantidad de planetas y portales.
2. Crear el grafo con planetas vértices.
3. Crear la tabla hash de ids.
4. Por cada portal:
 - leer planeta A, planeta B y costo
 - asignar id a A si no existe
 - asignar id a B si no existe
 - obtener ids numéricos
 - agregar arista ponderada al grafo
5. Leer origen y destino.
6. Convertir origen y destino a ids.
7. Ejecutar Dijkstra.
8. Imprimir el costo mínimo.

Complejidad

Sea:

N = cantidad de planetas / vértices

M = cantidad de portales / aristas

Construcción del grafo: $O(N + M)$ con hash promedio $O(1)$ y listas con add en $O(1)$.

Dijkstra: $O((N + M) \log M)$

También puede expresarse como: $O((N + M) \log N)$ para grafos simples.

Espacio: $O(N + M)$ por el grafo, la tabla hash, los arrays auxiliares y el heap.

Ejercicio 5

Para obtener la solución entendimos que necesitábamos verificar si un grafo no dirigido es bipartito. Un grafo es bipartito si puede colorearse con solo dos colores de forma que ninguna arista conecta dos vértices del mismo color.

La estrategia fue recorrer el grafo por componentes conexos usando BFS. Cada vez que se inicia una componente nueva, se pinta el primer vértice con color 0 y sus vecinos con el color opuesto, **1 - color[v]**.

Si durante el recorrido se encuentra una arista entre dos vértices que ya tienen el mismo color, el grafo no es bipartito y se imprime NO. Si se termina de revisar todos los vértices sin conflicto, se imprime SI.

TADs usados

Se utilizan los siguientes tipos de datos y estructuras:

- **adj_list_graph**: grafo representado con listas de adyacencia.
- **cola<int>**: cola FIFO para el recorrido BFS..
- **Iterator<edge>**: iterador para recorrer los vecinos de un vértice.

También se uso el array **int *color** para almacenar el color de cada vértice.

Flujo

1. Leer la cantidad de vértices y aristas.
2. Crear el grafo g con vértices y no dirigido.
3. Leer cada arista y agregarla al grafo con `g.addEdge(from, to)`.
4. Crear un arreglo color de tamaño vértices + 1 y asignar -1 a todos los vértices para indicar que están sin colorear.
5. Recorrer todos los vértices.
 - Si un vértice aún no tiene color (`color[i] == -1`), significa que empieza una nueva componente conexa.
 - Pintar ese vértice con 0.
 - Encolar el vértice en c y recorrer la componente con un bucle BFS.
6. Mientras la cola no esté vacía:
 - Sacar un vértice v de la cola.
 - Obtener sus vecinos mediante `g.neighbors(v)`.
 - Para cada vecino w:
 - Si w no está coloreado, asignarle `1 - color[v]` y encolarlo.
 - Si w ya está coloreado y `color[w] == color[v]`, entonces hay un conflicto y el grafo no es bipartito.
7. Si se detecta un conflicto, imprimir NO, liberar memoria y terminar.
8. Si se procesan todas las componentes sin conflictos, imprimir SI y liberar memoria.

Complejidad

Sea V la cantidad de vértices y E la cantidad de aristas.

- Construcción del grafo: $O(E)$.
- Inicialización del arreglo de colores: $O(V)$.
- Recorrido BFS en todo el grafo: $O(V + E)$.

En total:

$O(V + E)$

Declaración de autoría

El único momento en que usamos inteligencia artificial fue en el ejercicio 4. En este ejercicio teníamos un problema grande de rendimiento: las pruebas con 50.000 y 100.000 vértices demoraban aproximadamente 15 y 45 segundos respectivamente.

Primero revisamos por nuestra cuenta la complejidad del algoritmo. Como usábamos lista de adyacencia y Dijkstra con cola de prioridad, el orden esperado era: $O((M+N)\log N)$

Por lo tanto, el problema no parecía estar en Dijkstra. Al no encontrar la causa, recurrimos a IA, que nos sugirió revisar la función de hash y el tamaño de la tabla. Luego de cambiar la función de hash, ambas pruebas pasaron a ejecutarse en menos de 1 segundo.

El problema principal estaba en nuestra función de hash para strings:

```
virtual int hash(const char *data) override {  
    int i = 0;  
    int sum = 0;  
    while (data[i] != '\0'){  
        sum += data[i];  
        i++;  
    }  
    return sum;  
}
```

Esta función simplemente suma los valores ASCII de los caracteres. Por eso, strings con los mismos caracteres en distinto orden generan el mismo hash. Por ejemplo, "ola" y "alo" caen en la misma posición.

En nuestro caso, esto era mucho más grave porque los vértices tenían nombres muy parecidos, como planeta12345, planeta21345, planeta32145, etc. Como todos compartían el prefijo "planeta" y la función solo sumaba caracteres, el hash dependía de la suma de los dígitos del número final

Al hacer el cálculo, para los nombres desde planeta1 hasta planeta50000, nuestra función generaba solamente alrededor de 130 valores de hash distintos. Por lo tanto, había aproximadamente: $50000 - 130 = 49870$ colisiones iniciales.

Para planeta1 hasta planeta100000, generaba solamente alrededor de 136 valores de hash distintos. Entonces había aproximadamente: $100000 - 136 = 99864$ colisiones iniciales.

Esto hacía que operaciones como contains, set y get, que deberían comportarse casi como $O(1)$, se volvieran mucho más lentas en la práctica.

En conclusión, el problema no era Dijkstra, sino la función de hash usada para convertir los nombres de planetas en IDs. Al cambiarla por una función que distribuye mejor los strings, el rendimiento mejoró drásticamente.

La inteligencia artificial nos daba una función de string muy rara, pero justo estábamos con un compañero de clase, Nicolas Faccio, y nos comentó la idea que tuvieron ellos para la función de hash con string: recorrer el string carácter por carácter y en cada paso, multiplicar el valor acumulado por algún número primo y luego sumar el valor ASCII del carácter actual.